

Task 03 – Integration Design

Overview

To ensure reliability, scalability, and low latency, I would implement the integration using a **custom Node.js/Express backend** with the **HubSpot CRM API**, **Karix WhatsApp Business API**, and **Google Ads Conversion Tracking API**. A custom backend provides greater control over validation, error handling, retries, and security compared to no-code tools such as Zapier or Make.

End-to-End Architecture

1. A patient submits the consultation form on the landing page.
2. The frontend sends the form data (Name, Phone, Clinic Preference) to the backend via a secure HTTPS API.
3. The backend validates and sanitizes the input.
4. The backend searches HubSpot using the **CRM Search API** with the **phone number** as the lookup field.
5. If a contact with the same phone number exists, the record is updated.
6. Otherwise, a new contact is created.
7. While creating or updating the contact, the backend also sets:
 8. **Source:** Google Ads - Consultation Landing Page
 9. **Lead Status:** New Enquiry
10. After HubSpot confirms a successful operation:
11. A request is sent to the **Karix WhatsApp Business API** to send a confirmation message.
12. The **Google Ads conversion** (`consultation_form_submitted`) is triggered so campaign optimization is based on successful enquiries.

Why This Approach?

Although HubSpot provides native forms and automation tools, a custom backend is more suitable because it offers complete control over business logic, validation, retries, logging, and third-party integrations. It is also easier to extend in the future if additional systems need to be integrated.

Biggest Failure Point

The most critical failure point is the HubSpot integration. HubSpot's default deduplication works on **email**, whereas this form collects only a **phone number**. To prevent duplicate contacts, the backend must always search for an existing contact using the phone number before deciding whether to create or update a record.

If HubSpot is temporarily unavailable, each submission should first be stored in a durable queue (such as AWS SQS or RabbitMQ). Failed requests can then be retried automatically with exponential backoff, while persistent failures are moved to a dead-letter queue for manual investigation. This ensures that no patient enquiry is lost.

WhatsApp SLA (Within 2 Minutes)

The two-minute SLA could be affected by:

- HubSpot API downtime
- Queue congestion
- Backend server issues
- Network failures
- Karix API latency

To monitor this SLA, I would log timestamps for:

- Form submission
- HubSpot processing
- WhatsApp API request
- WhatsApp API response

Application monitoring and alerts would notify the team if delivery exceeds two minutes or if API error rates increase, allowing quick identification and resolution of issues.